IBM FileNet P8 V5.5.8

Detailed Study Material

Day 9 – API Integration & Automation

FileNet APIs | Key Operations | Event-Driven Automation | REST & Java Integration

Author	Course Day	Version
Rajesh Pasham	**Day 9 of 10**	**IBM FileNet P8 V5.5.8**

Section 1 | Learning Objectives

By the end of Day 9, students will be able to:

1	Describe the FileNet P8 API landscape: Java CEI API, .NET API, and Content Engine REST API
2	Understand the Content Engine Java API object model: Factory, Domain, ObjectStore, Document
3	Write Java code to connect to Content Engine V5.5.8, create documents, and update metadata
4	Use the FileNet P8 REST API (CMIS and CE REST) to perform CRUD operations via HTTP
5	Construct and execute SPARQL/SQL search queries against the object store via API
6	Create and configure Event Subscriptions in ACCE to trigger automation on document events
7	Implement Pre-processing and Post-processing event handlers using Content Engine EJB events
8	Develop a Custom Event Handler (IObjectChangeEventHandler) using the CE Java API
9	Execute REST API calls using curl and Postman to upload documents to IBM FileNet V5.5.8
10	Trigger workflow launch using the Process Engine REST API

Section 2 | FileNet API Landscape

2 FileNet P8 API Landscape

IBM FileNet P8 V5.5.8 exposes content management capabilities through three primary API families. Each serves different integration patterns, programming languages, and deployment models.

API Type	Description & Use Case
Java Content Engine API (CEI)	Full-featured Java API for deep integration. Direct EJB or IIOP connection to CE. Full object model access.
.NET / COM API	Windows client API wrapping the Java CEI. Used in .NET enterprise applications. Feature parity with Java API.
Content Engine REST API	Stateless HTTP/JSON API for browser, mobile, and cross-platform integrations. Based on CMIS and CE extensions.
CMIS (oData/AtomPub)	Open standard protocol for content repositories. Accessible via /fncmis endpoint on WAS. Interoperability focus.
Process Engine REST API	HTTP API for workflow operations: launch, step, query work items. Used by ICN and custom workflow UIs.

2.1 Java Content Engine API (CEI)

The Java CEI API is the most powerful and complete API for FileNet P8. It exposes the full object model and is used by IBM products (ICN, ACCE) themselves internally.

2.1.1 Object Model Hierarchy

Class / Interface	Description
UserContext	Thread-level security context. Holds the authenticated Subject. Must be set before any CE operation.
Domain	Top-level object representing the FileNet P8 domain (P8Domain). Entry point for all API operations.
ObjectStore	Represents a specific object store (e.g., TrainingOS). All documents and folders exist within an ObjectStore.
Document	Core content object. Has properties (metadata), content elements (binary files), and version series.
Folder	Container for documents. Supports nested hierarchy. Has its own ACL.
PropertyFilter	Controls which properties are fetched from the server. Use to optimise network I/O.
SearchSQL / SearchScope	Used to construct and execute CEI SQL search queries against an ObjectStore.
Factory	Static factory class for creating new instances: Document.Factory, Folder.Factory, etc.

2.1.2 Required JAR Files (FileNet P8 V5.5.8)

JAR Files Required for Java CEI API Development

From the CE installation or client installer, add these JARs to your project classpath:

Jace.jar – Main CE Java API library (Content Engine client)
stax-api.jar – XML Streaming API (required by Jace)
log4j.jar – Logging framework (required by Jace)
javaee.jar – J2EE API (for EJB transport layer)
activation.jar – JavaBeans Activation Framework

Location on server: C:\IBM\FileNet\ContentEngine\lib\
Download path: CE_CLIENT_INSTALL_DIR\lib\Jace.jar

2.1.3 Connection Methods

Connection Type	Description & URL
EJB (IIOP)	Direct EJB connection over IIOP protocol. Best for server-side Java apps on same WAS. URL: iiop://localhost:2809
MTOM (Web Services)	SOAP/MTOM web service connection. Firewall-friendly. URL: http://localhost:9080/wsi/FNCEWS40MTOM/
Local EJB	For code running inside the same WAS JVM as CE. No network overhead. Uses local EJB interface.

2.2 Content Engine REST API

The CE REST API (introduced formally in V5.2+) provides a stateless, HTTP-based interface to FileNet content operations. In FileNet P8 V5.5.8 it is accessible under the /fncmis (CMIS) and /CE_API (CE REST) contexts.

2.2.1 REST API Base URLs (Training Lab)

Endpoint	URL
CE REST API Base	http://localhost:9080/CE_API/P8Domain/
CMIS AtomPub Base	http://localhost:9080/fncmis/resources/
CMIS JSON Browser	http://localhost:9080/fncmis/browser/
Swagger / API Explorer	http://localhost:9080/CE_API/docs/ (V5.5.8+)
Process REST API	http://localhost:9080/pe-rest/v1/

2.2.2 Authentication for REST API

The CE REST API uses HTTP Basic Authentication or LTPA token-based authentication:

- Basic Auth: Base64-encode username:password in Authorization header. Example: Authorization: Basic Y2VhZG1pbjpjZWFkbWlu
- LTPA Token: Obtain token via POST /j_security_check, then pass in Cookie header for subsequent calls
- For development/testing: Use Basic Auth with ceadmin:ceadmin credentials
- For production: Configure LTPA SSO or use OAuth 2.0 (available in newer versions via CP4BA)

2.2.3 REST API Resource Model

Method & Path	Purpose	Notes
GET /repositories	List all repositories (object stores)	No request body. Returns JSON array of OS descriptors.
GET /repositories/{osname}/objects/{id}	Retrieve object by ID	Returns full property set as JSON.
POST /repositories/{osname}/objects	Create new document or folder	JSON body with properties + multipart for content.
PUT /repositories/{osname}/objects/{id}	Update object properties	JSON body with changed properties.
DELETE /repositories/{osname}/objects/{id}	Delete object	No body. Returns 204 on success.
POST /repositories/{osname}/searches	Execute a search query	JSON body with SQL query string.

2.3 .NET API Overview

The IBM FileNet .NET API (also called the Content Manager .NET API) is a Windows-only wrapper around the Java CEI that allows .NET developers to work with FileNet using C# or VB.NET.

Key Points for .NET API

- Installed via IBM FileNet P8 Client Install on Windows (adds .NET assemblies to GAC).
- Main assembly: FileNet.Api.dll — found in C:\Program Files\IBM\FileNet\ContentEngine\Client\
- Object model mirrors Java CEI: Factory, Domain, ObjectStore, Document, Folder.
- Connection transport: MTOM web services (same URL as Java MTOM mode).
- Not covered in lab exercises (lab environment is Linux/WAS-based), but concept is tested in IBM C1000-074 exam.
- C# sample connection: IConnectionFactory cf = ConnectionFactory.GetInstance().GetConnection("http://host:9080/wsi/FNCEWS40MTOM/", null);

Section 3 | Key Operations via Java CEI API

3 Key Operations via Java CEI API

This section provides complete, working Java code examples for every key FileNet operation. All examples are written for FileNet P8 V5.5.8 using the MTOM web services transport (port 9080).

Standard Connection Setup (used in all examples below)

The following code establishes a connection to the CE and sets up the UserContext.
Include this block at the start of every Java program or test class:

```
import com.filenet.api.core.*;
import com.filenet.api.util.UserContext;
import com.filenet.api.property.*;
import com.filenet.api.query.*;
import com.filenet.api.collection.*;
import com.filenet.api.exception.*;
import javax.security.auth.Subject;
import java.util.*;

public class FileNetBase {
    // Training lab connection parameters
    static final String CE_URI = "http://localhost:9080/wsi/FNCEWS40MTOM/";
    static final String USERNAME = "ceadmin";
    static final String PASSWORD = "ceadmin";
    static final String OS_NAME  = "TrainingOS";

    public static Connection getConnection() {
        return Factory.Connection.getConnection(CE_URI);
    }

    public static Domain getDomain(Connection conn, Subject subject) {
        UserContext.get().pushSubject(subject);
        return Factory.Domain.fetchInstance(conn, null, null);
    }

    public static ObjectStore getObjectStore(Domain domain) {
        return Factory.ObjectStore.fetchInstance(domain, OS_NAME, null);
    }

    public static Subject getSubject(Connection conn) {
        return UserContext.createSubject(conn, USERNAME, PASSWORD, "FileNetP8");
    }
}
```

3.1 Document Creation

Creating a document in FileNet requires: (1) creating a Document instance, (2) setting properties, (3) attaching content, and (4) filing it into a folder.

```
// Document Creation - FileNet P8 V5.5.8
public static void createDocument(ObjectStore os, String folderPath, String
filePath) throws Exception {
    // Step 1: Create a new Document instance of class 'Document'
    Document doc = Factory.Document.createInstance(os, "Document");

    // Step 2: Set metadata properties
    doc.getProperties().putValue("DocumentTitle", "API Created Document");
    doc.getProperties().putValue("DocumentDescription", "Created via Java CEI
API");

    // Step 3: Attach binary content
    ContentElementList contentList = Factory.ContentElement.createList();
    ContentTransfer ct = Factory.ContentTransfer.createInstance();
    java.io.InputStream is = new java.io.FileInputStream(filePath);
    ct.setCaptureSource(is);
    ct.set_RetrievalName(new java.io.File(filePath).getName());
    ct.set_ContentType("application/pdf");
    contentList.add(ct);
    doc.set_ContentElements(contentList);

    // Step 4: Set checkin type (MAJOR_VERSION creates a published version)
    doc.checkin(AutoClassify.DO_NOT_AUTO_CLASSIFY,
               CheckinType.MAJOR_VERSION);

    // Step 5: Save the document to the object store
    doc.save(RefreshMode.REFRESH);

    // Step 6: File the document into a folder
    Folder folder = Factory.Folder.fetchInstance(os, folderPath, null);
    ReferentialContainmentRelationship rcr =
        folder.file(doc, AutoUniqueName.AUTO_UNIQUE,
                    "API Created Document",
                    DefiningObjectStore.TOGETHER_WITH_CONTENT);
    rcr.save(RefreshMode.NO_REFRESH);
    System.out.println("Document created with ID: " + doc.get_Id());
}
```

3.2 Metadata Updates

Updating document metadata involves fetching the document instance and using getProperties().putValue() to update property values before saving.

```
// Metadata Update - FileNet P8 V5.5.8
public static void updateMetadata(ObjectStore os, String documentId) {
    // Fetch document by ID
    Document doc = Factory.Document.fetchInstance(os,
        new com.filenet.api.util.Id(documentId), null);

    // Update standard and custom properties
    Properties props = doc.getProperties();
    props.putValue("DocumentTitle", "Updated Title via API");

    // For custom String property:
    // props.putValue("CustomStringProp", "NewValue");
```

```
    // For custom Date property:
    // props.putValue("CustomDateProp", new java.util.Date());

    // Save with NO_REFRESH for performance (no round-trip fetch after save)
    doc.save(RefreshMode.NO_REFRESH);
    System.out.println("Metadata updated for: " + documentId);
}
```

3.3 Folder Operations

Folder operations include creating folders, filing documents, and navigating the folder hierarchy.

```
// Folder Operations - FileNet P8 V5.5.8
public static Folder createFolder(ObjectStore os, String parentPath, String
folderName) {
    // Fetch parent folder
    Folder parentFolder = Factory.Folder.fetchInstance(os, parentPath, null);

    // Create child folder
    Folder newFolder = Factory.Folder.createInstance(os, null);
    newFolder.set_FolderName(folderName);
    newFolder.set_Parent(parentFolder);
    newFolder.save(RefreshMode.REFRESH);

    System.out.println("Folder created: " + newFolder.get_PathName());
    return newFolder;
}

// List documents in a folder
public static void listFolderContents(ObjectStore os, String folderPath) {
    Folder folder = Factory.Folder.fetchInstance(os, folderPath, null);
    DocumentSet docs = folder.get_ContainedDocuments();
    java.util.Iterator<?> iter = docs.iterator();
    while (iter.hasNext()) {
        Document doc = (Document) iter.next();
        System.out.println("  - " +
doc.getProperties().getStringValue("DocumentTitle"));
    }
}
```

3.4 Search Queries

FileNet P8 supports SQL-92 style queries called CE SQL (sometimes called CBSQL). These can be executed via the Java API using SearchSQL and SearchScope classes.

3.4.1 CE SQL Syntax

CE SQL Query	Purpose
SELECT * FROM Document WHERE DocumentTitle LIKE '%API%'	Find all documents with 'API' in the title

SELECT * FROM Document WHERE This INFOLDER '/Training/Docs'	Find documents filed in a specific folder
SELECT * FROM Document WHERE DateCreated > TIMESTAMP '2024-01-01T00:00:00Z'	Find documents created after a date
SELECT d.Id, d.DocumentTitle FROM Document d WHERE d.DocumentClass = 'HR_Document'	Select specific properties from a custom class
SELECT * FROM Document WHERE ContentSize > 1048576	Find documents larger than 1 MB

```
// Search Query - FileNet P8 V5.5.8
public static void searchDocuments(ObjectStore os, String sqlQuery) {
    SearchSQL sql = new SearchSQL(sqlQuery);
    SearchScope scope = new SearchScope(os);

    // PropertyFilter controls which properties are returned
    PropertyFilter pf = new PropertyFilter();
    pf.addIncludeProperty(new FilterElement(null, null, null, "DocumentTitle",
null));
    pf.addIncludeProperty(new FilterElement(null, null, null, "Id", null));
    pf.addIncludeProperty(new FilterElement(null, null, null, "DateCreated",
null));

    IndependentObjectSet results = scope.fetchObjects(sql, 100, pf, Boolean.TRUE);
    java.util.Iterator<?> iter = results.iterator();
    int count = 0;
    while (iter.hasNext()) {
        Document doc = (Document) iter.next();
        System.out.println("Found: " +
doc.getProperties().getStringValue("DocumentTitle")
            + " | ID: " + doc.get_Id());
        count++;
    }
    System.out.println("Total results: " + count);
}
```

Section 4 | REST API Operations

4 REST API Operations

The FileNet P8 REST API and CMIS API allow integration without compiling Java code. This section covers common operations using curl commands compatible with FileNet P8 V5.5.8.

4.1 List Object Stores (CMIS)

```
# Get list of repositories (object stores) via CMIS AtomPub
curl -u ceadmin:ceadmin \
     -H "Accept: application/json" \
     "http://localhost:9080/fncmis/browser?cmisselector=repositories"

# Expected response: JSON array with repository info including repositoryId
# repositoryId for TrainingOS will be something like: {GUID}
```

4.2 Upload Document via CMIS Browser Binding

```
# Upload a document to the Root Folder of TrainingOS
# Replace {REPOSITORY_ID} with the actual GUID from step 4.1

curl -u ceadmin:ceadmin \
     -X POST \
     -H "Accept: application/json" \
     -F "cmisaction=createDocument" \
     -F "propertyId[0]=cmis:name" \
     -F "propertyValue[0]=REST_Uploaded_Document.pdf" \
     -F "propertyId[1]=cmis:objectTypeId" \
     -F "propertyValue[1]=cmis:document" \
     -F "content=@/tmp/test_document.pdf;type=application/pdf" \
     "http://localhost:9080/fncmis/browser/{REPOSITORY_ID}/root"

# To file into a specific folder, replace /root with the folder path:
# "http://localhost:9080/fncmis/browser/{REPOSITORY_ID}/Training/Docs"
```

4.3 Get Document Properties

```
# Get document properties by CMIS object ID
curl -u ceadmin:ceadmin \
     -H "Accept: application/json" \

"http://localhost:9080/fncmis/browser/{REPOSITORY_ID}?cmisselector=object&objectId={DOC_I
D}"

# Get document by path
curl -u ceadmin:ceadmin \
     -H "Accept: application/json" \
```

```
"http://localhost:9080/fncmis/browser/{REPOSITORY_ID}?cmisselector=object&objectId=/Train
ing/Docs/MyDoc.pdf"
```

4.4 Execute Search via CMIS Query

```
# Execute a CMIS query (maps to CE SQL internally)
curl -u ceadmin:ceadmin \
     -X POST \
     -H "Content-Type: application/x-www-form-urlencoded" \
     -H "Accept: application/json" \
     -d 'cmisaction=query' \
     -d
'statement=SELECT+*+FROM+cmis:document+WHERE+cmis:name+LIKE+%27%25API%25%27' \
     -d 'maxItems=20' \
     "http://localhost:9080/fncmis/browser/{REPOSITORY_ID}"
```

4.5 Process Engine REST API – Launch Workflow

```
# Step 1: Get available workflow definitions
curl -u ceadmin:ceadmin \
     -H "Accept: application/json" \
     "http://localhost:9080/pe-
rest/v1/workobjects/queueitems?queueName=DefaultRoster"

# Step 2: Launch a workflow instance
curl -u ceadmin:ceadmin \
     -X POST \
     -H "Content-Type: application/json" \
     -H "Accept: application/json" \
     -d '{
           "wfDefinition": "TrainingWorkflow",
           "launchStep": "Launch",
           "data": {
             "F_Subject": "API Triggered Workflow",
             "DocumentID": "{DOCUMENT_GUID}"
           }
         }' \
     "http://localhost:9080/pe-rest/v1/workobjects"
```

Section 5 | Event-Driven Automation

5 Event-Driven Automation

FileNet P8 provides a powerful event-driven automation framework that allows you to trigger server-side logic automatically when content operations occur — without modifying the calling application.

5.1 Event Subscription Architecture

The event system works through three components: Events, Subscriptions, and Event Handlers.

Component	Description	Key Note
Event	A predefined action that occurs on a FileNet object. Examples: OnCreate, OnCheckin, OnDelete, OnClassify.	Cannot be disabled. Always fired when the action occurs.
Subscription	Configuration that links a specific Event on a specific Class/Folder to an Event Action.	Configured in ACCE per object store. Can be enabled/disabled.
Event Action	The code or process that is invoked when the Subscription fires. Types: Code Module, Email, Process Router.	Code Module = custom Java class. Process Router = workflow launch.

5.2 Event Types in FileNet P8 V5.5.8

Event Name	Description
OnCreation	Fires when a new object (document, folder, annotation) is created in the object store.
OnCheckin	Fires when a document version is checked in (major or minor version checkin).
OnDelete	Fires when an object is permanently deleted from the object store.
OnClassify	Fires when a document is auto-classified by IBM Content Collector.
OnPromote	Fires when a document version is promoted from minor to major.
OnDemote	Fires when a document version is demoted from major to minor.
OnChangeState	Fires when the security (ACL) of an object is modified.
OnFolderCreation	Fires when a folder object is created.

5.3 Pre-Processing and Post-Processing

Every event subscription can be configured with Pre-processing (fires before the action completes) or Post-processing (fires after the action completes) behaviour.

Pre-processing vs Post-processing

PRE-PROCESSING (synchronous):
- Fires BEFORE the triggering operation is committed to the database.
- Can VETO the operation by throwing an exception (operation is rolled back).
- Used for: validation, mandatory metadata checks, security pre-screening.
- Risk: If handler throws exception, the original operation fails for the end user.

POST-PROCESSING (asynchronous):
- Fires AFTER the triggering operation is committed.
- Cannot veto the operation (it has already happened).
- Used for: notifications, workflow launches, audit logging, replication.
- More resilient: failures in handler do not affect the original operation.

5.4 Custom Event Handler Implementation

A Custom Event Handler is a Java class that implements the IObjectChangeEventHandler interface from the CE API. The compiled class is packaged as a JAR and uploaded to FileNet as a Code Module.

5.4.1 Event Handler Interface

```
import com.filenet.api.events.*;
import com.filenet.api.core.*;
import com.filenet.api.exception.*;

/**
 * Custom Event Handler for FileNet P8 V5.5.8
 * Fires on document creation - logs metadata to server console
 */
public class DocumentCreationHandler implements IObjectChangeEventHandler {

    @Override
    public void onEvent(ObjectChangeEvent event, Id subscriptionId) {
        try {
            // Get the object that triggered the event
            IndependentlyPersistableObject obj = event.get_Object();

            if (obj instanceof Document) {
                Document doc = (Document) obj;
                String title = doc.getProperties().getStringValue("DocumentTitle");
                String id = doc.get_Id().toString();

                // Log to FileNet server console (SystemOut.log)
                System.out.println("[EventHandler] Document created: " + title + "
ID: " + id);

                // Example: Enforce mandatory metadata
                if (title == null || title.isEmpty()) {
                    throw new EngineRuntimeException(
```

```
                        new ExceptionContext(
                            ExceptionCode.E_NOT_SUPPORTED,
                            "DocumentTitle is mandatory. Document rejected.",
                            null));
                }
            }
        } catch (EngineRuntimeException e) {
            throw e;  // Re-throw to veto if pre-processing
        } catch (Exception e) {
            System.err.println("[EventHandler] Error: " + e.getMessage());
        }
    }
}
```

5.4.2 Deploying the Event Handler as a Code Module

1. Compile the event handler Java class with Jace.jar on the classpath:
   `javac -cp C:\IBM\FileNet\ContentEngine\lib\Jace.jar DocumentCreationHandler.java`
2. Package into a JAR file:
   `jar cvf DocumentCreationHandler.jar DocumentCreationHandler.class`
3. In ACCE, navigate to: Object Stores > TrainingOS > Events, Actions, Processes > Code Modules
4. Click New Code Module. Upload the DocumentCreationHandler.jar file.
5. Navigate to: Events, Actions, Processes > Event Actions
6. Click New Event Action. Set Type = 'Code Module'. Select the uploaded Code Module.
7. Navigate to: Events, Actions, Processes > Subscriptions
8. Click New Subscription. Set: Event Class = OnCreation, Object Class = Document (or specific class).
9. Under Event Action, select the Event Action created in step 7.
10. Set Timing: Pre-processing (to enable veto) or Post-processing.
11. Enable the subscription and click Save.

5.5 Process Router – Launching Workflows from Events

The most common event action in production FileNet systems is the Process Router, which automatically launches a workflow when a document event fires.

Process Router Configuration Steps

1. Ensure Process Engine is installed and a workflow definition (e.g., TrainingWorkflow) is deployed.
2. In ACCE, navigate to: Events, Actions, Processes > Event Actions > New Event Action.
3. Set Action Type = 'Process Router'. This reveals the Process Router configuration panel.
4. Set Workflow Definition Name: TrainingWorkflow
5. Set Launch Step: LaunchStep (the entry step in the workflow)
6. Map data fields: F_Subject = EventObject.DocumentTitle (maps document title to workflow subject).
7. Create a Subscription linking OnCheckin event on your document class to this Event Action.
8. Test: Check in a document of the subscribed class. A new workflow instance should appear in the queue.

Section 6 | Troubleshooting & Quick Reference

6 Troubleshooting & Quick Reference

6.1 Common API Errors

Error	Cause	Resolution
E_OBJECT_NOT_FOUND	Object ID or path does not exist in the object store	Verify the ID/path. Check the object store name in the URL.
E_ACCESS_DENIED	Authenticated user lacks permission on the target object	Add the user/group to the object ACL in ACCE with required permissions.
E_NOT_UNIQUE	Duplicate symbolic name or unique key violation	Change the symbolic name. Check for duplicates in ACCE.
401 Unauthorized (REST)	Invalid credentials in Authorization header	Verify Basic Auth encoding. Check ceadmin password in WAS.
404 Not Found (REST)	Wrong CMIS URL or repository ID	Verify /fncmis/browser endpoint and the repositoryId GUID.
Connection refused IIOP	CE server not running or wrong port	Verify WAS is started. Check IIOP port 2809 in WAS Admin Console.
EngineRuntimeException	Various CE API errors	Check the ExceptionCode for specific details. Review SystemOut.log.

6.2 Key URLs and Parameters – Training Lab

Resource	Value
CE MTOM Web Services	http://localhost:9080/wsi/FNCEWS40MTOM/
CMIS Browser Binding	http://localhost:9080/fncmis/browser/
CMIS AtomPub	http://localhost:9080/fncmis/resources/
Process Engine REST	http://localhost:9080/pe-rest/v1/
ACCE Admin Console	http://localhost:9080/acce
WAS Admin Console	https://localhost:9043/ibm/console
CE Admin Credentials	ceadmin / ceadmin
WAS Credentials	wasadmin / wasadmin
Object Store Name	TrainingOS
Domain Name	P8Domain
CE Log (SystemOut)	C:\IBM\WAS\profiles\FileNetServer\logs\server1\SystemOut.log